MERN Practical Codes — Step-by-Step Guide

28 questions • 6 master-code sections • Follow the steps in order, without guessing missing setup steps

Before you start: one-time setup

1. Open VS Code. Create a main folder named `MERN_Practicals` and open that folder in VS Code.

2. For backend practicals, create a folder named `Backend`. Open a terminal in that folder (Terminal → New Terminal).

3. In the Backend terminal, run `npm init -y` once. Then run `npm install express mongoose cors` once.

4. Create `server.js` inside Backend and paste the Master 2 code into it. Save with Ctrl+S.

5. Make sure MongoDB is running if your practical uses MongoDB. MongoDB Compass is only the visual interface; the MongoDB server itself must be running.

6. To create the React frontend, open a second terminal at the main `MERN_Practicals` folder and run `npm create vite@latest Frontend`. Select React → JavaScript.

7. Run `cd Frontend`, then `npm install`, then `npm install axios`. For Master 6 also run `npm install react-router-dom`.

8. Replace the contents of `Frontend/src/App.jsx` with the relevant React master code. If the code imports `./App.css`, create/update `Frontend/src/App.css` too.

9. Keep the backend and frontend in separate terminals. Do not close the backend terminal while using the React app.

Folder layout

```
MERN_Practicals/
  Backend/
    package.json
    server.js
  Frontend/
    package.json
    src/
      App.jsx
      App.css   (only if needed)
```

Important: If you already created the folders and installed packages, do not repeat the setup commands every time. For each new practical, replace the relevant code, save it, and follow that practical's run steps.

MASTER 1 — JavaScript Basics

Questions covered: operators/control statements, even numbers, addition, factorial, arrow functions, multiplication table, and map/filter/reduce.

```
// 1. Operators and control statements
let a = 10, b = 5;
console.log("Sum:", a + b);
if (a > b) console.log("a is greater");
for (let i = 1; i <= 5; i++) console.log(i);

// 2. Print even numbers from 1 to 50
for (let i = 2; i <= 50; i += 2) console.log(i);

// 3. Add two numbers
let x = 10, y = 20;
console.log("Sum =", x + y);

// 4. Factorial
let n = 5, fact = 1;
for (let i = 1; i <= n; i++) fact *= i;
console.log("Factorial =", fact);

// 5. Arrow function
const square = n => n * n;
console.log(square(5));

// 6. Multiplication table
let num = 5;
for (let i = 1; i <= 10; i++)
  console.log(num + " x " + i + " = " + num * i);

// 7. map(), filter(), reduce()
let nums = [1, 2, 3, 4, 5];
console.log(nums.map(n => n * 2));
console.log(nums.filter(n => n % 2 === 0));
console.log(nums.reduce((sum, n) => sum + n, 0));
```

Exact steps to perform

1. In VS Code, create a file named `script.js` in your practical folder.

2. Paste the code above into `script.js` and save it.

3. Open a terminal in the same folder. Make sure Node.js is installed by running `node -v`.

4. Run `node script.js`.

5. Read the output in the terminal. If the examiner asks for only one program, show only that program's relevant part and run it.

Remember

1. This is a JavaScript-only practical. You do not need Express, MongoDB, or the React development server for these questions.

2. To demonstrate another number, change the values of n or `num`, save, and run the command again.

MASTER 2 — Express + Mongoose Backend (`Backend/server.js`)

Questions covered: server response, Express routes, JSON response, Express API, GET API, MongoDB insert/retrieve/update/delete. This follows the backend pattern in your college Practical 10.

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/CollegeDB")
  .then(() => {
    console.log("MongoDB Connected Successfully");
    app.listen(3000, () => console.log("Server running on port 3000"));
  })
  .catch(err => console.log("MongoDB Connection Failed:", err.message));

const Student = mongoose.model("Student", {
  name: { type: String, required: true, trim: true },
  course: { type: String, required: true, trim: true },
  marks: { type: Number, required: true, min: 0, max: 100 }
});

app.get("/", (req, res) => res.send("Hello from Express Server!"));
app.get("/home", (req, res) => res.send("Home Page"));
app.get("/about", (req, res) => res.send("About Page"));
app.get("/json", (req, res) => res.json({ message: "Hello", status: "success" }));
app.get("/api", (req, res) => res.json({ message: "API working" }));

app.post("/students", async (req, res) => {
  try {
    const student = await Student.create(req.body);
    res.status(201).json(student);
  } catch (err) {
    res.status(400).json({ message: "Invalid student data" });
  }
});

app.get("/students", async (req, res) => {
  try {
    res.json(await Student.find());
  } catch (err) {
    res.status(500).json({ message: "Unable to fetch students" });
  }
});

app.put("/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndUpdate(
      req.params.id, req.body, { new: true, runValidators: true }
    );
    if (!student) return res.status(404).json({ message: "Student not found" });
    res.json(student);
  } catch (err) {
    res.status(400).json({ message: "Unable to update student" });
  }
});

app.delete("/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);
    if (!student) return res.status(404).json({ message: "Student not found" });
    res.json({ message: "Student deleted successfully" });
  } catch (err) {
    res.status(400).json({ message: "Unable to delete student" });
  }
});
```

Exact steps to perform

1. Start MongoDB Server. Open MongoDB Compass if you want to view the database; connect to `mongodb://127.0.0.1:27017`.

2. Open the `Backend` folder in a VS Code terminal.

3. If this is the first time only, run `npm init -y`, then `npm install express mongoose cors`.

4. Create/open `server.js`, paste the Master 2 code, and save it.

5. In the Backend terminal, run `node server.js`.

6. Wait for both messages: `MongoDB Connected Successfully` and `Server running on port 3000`. Keep this terminal running.

7. Open `http://localhost:3000/` in your browser to test the message route. Test `/home`, `/about`, `/json`, and `/api` by changing the end of the address.

8. For POST/PUT/DELETE, use Postman. For POST, choose Body → raw → JSON and send `name`, `course`, and `marks` to `http://localhost:3000/students`.

9. Use GET `http://localhost:3000/students` to see records. Use PUT `/students/RECORD_ID` to update and DELETE `/students/RECORD_ID` to delete. Replace RECORD_ID with a real MongoDB document ID.

10. For a frontend practical, leave this terminal running and open a second terminal for the Frontend steps in Master 5.

If you only need a simple Express route

1. You can still use this same running server. Open the route in the browser and use the path specified by the question.

2. Do not start another server on port 3000 while this one is already running. If you edit the code, stop it with Ctrl+C and run `node server.js` again.

MASTER 3 — React Components, Props, State and CSS (`Frontend/src/App.jsx`)

Questions covered: basic CSS, functional component, multiple components, props, and useState.

```
import { useState } from "react";
import "./App.css";

function Welcome() {
  return <h2>Welcome to React</h2>;
}

function Student(props) {
  return <p>Name: {props.name}</p>;
}

function App() {
  const [count, setCount] = useState(0);
  return (
    <div className="box">
      <h1>React Practical</h1>
      <Welcome />
      <Student name="Parth" />
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}

export default App;

/* Put this CSS in App.css:
.box {
  text-align: center;
  margin: 30px;
  padding: 20px;
  border: 1px solid black;
}
*/
```

Exact steps to perform

1. Open the `Frontend` folder in VS Code. If it has not been created, create it using the one-time setup instructions at the beginning of this PDF.

2. Open `src/App.jsx`, select all existing code, replace it with the Master 3 code, and save.

3. Create/open `src/App.css`. Copy only the CSS rules shown inside the comment into that CSS file, then save.

4. Open a second terminal in the Frontend folder (not the Backend folder).

5. Run `npm run dev`.

6. Wait for Vite to show a local address, usually `http://localhost:5173/`. Open that address in your browser.

7. Show the Welcome component, Student component receiving a prop, and the Increase button changing the count.

8. Keep the Vite terminal running while demonstrating the React page. This practical does not need the backend terminal.

MASTER 4 — React Dynamic Input and Date/Time (`Frontend/src/App.jsx`)

Questions covered: dynamic date/time and a form/input that displays user input immediately.

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  return (
    <div>
      <h2>Dynamic Input</h2>
      <input
        value={name}
        onChange={e => setName(e.target.value)}
        placeholder="Enter your name"
      />
      <p>Hello {name}</p>
      <p>Current date and time: {new Date().toLocaleString()}</p>
    </div>
  );
}

export default App;
```

Exact steps to perform

1. Open `Frontend/src/App.jsx`.

2. Replace all code in the file with Master 4 and save.

3. Open a terminal in the `Frontend` folder.

4. Run `npm run dev`. If Vite is already running in another terminal, you do not need to run it again.

5. Open the local Vite URL in the browser. Type into the input box; the greeting changes as you type.

6. The date/time is displayed using `new Date().toLocaleString()`. Refresh the page to render the latest time.

MASTER 5 — React Form + Axios + Backend + MongoDB (`Frontend/src/App.jsx`)

Questions covered: fetch data from API, display MongoDB records, submit form to backend, and validate input. The backend routes are in Master 2.

```
import { useEffect, useState } from "react";
import axios from "axios";

const API = "http://localhost:3000/students";

function App() {
  const [students, setStudents] = useState([]);
  const [name, setName] = useState("");
  const [course, setCourse] = useState("");
  const [marks, setMarks] = useState("");

  const getStudents = async () => {
    try {
      const response = await axios.get(API);
      setStudents(response.data);
    } catch (err) {
      alert("Unable to load students. Check that backend is running.");
    }
  };

  useEffect(() => { getStudents(); }, []);

  const addStudent = async e => {
    e.preventDefault();
    if (!name.trim() || !course.trim() || marks === "" ||
        !Number.isFinite(Number(marks)) ||
        Number(marks) < 0 || Number(marks) > 100) {
      alert("Enter valid details. Marks must be 0-100.");
      return;
    }

    try {
      await axios.post(API, {
        name: name.trim(),
        course: course.trim(),
        marks: Number(marks)
      });
      setName("");
      setCourse("");
      setMarks("");
      await getStudents();
    } catch (err) {
      alert("Unable to add student. Check backend/API.");
    }
  };

  return (
    <div>
      <h1>Student List</h1>
      <form onSubmit={addStudent}>
        <input placeholder="Name" value={name}
          onChange={e => setName(e.target.value)} />
        <input placeholder="Course" value={course}
          onChange={e => setCourse(e.target.value)} />
        <input type="number" placeholder="Marks (0-100)"
          min="0" max="100" value={marks}
          onChange={e => setMarks(e.target.value)} />
        <button type="submit">Add Student</button>
      </form>
      {students.map(s => (
        <p key={s._id}>{s.name} — {s.course} — {s.marks}</p>
      ))}
    </div>
  );
}

export default App;
```

Exact steps to perform — do these in order

1. **Backend terminal:** open the `Backend` folder and run `node server.js`. Wait until MongoDB and port 3000 connection messages appear. Keep this terminal running.
2. **Frontend setup (first time only):** open a second terminal at `Frontend` and run `npm install`, then `npm install axios`.
3. Open `Frontend/src/App.jsx`, replace all code with Master 5, and save.
4. **Frontend terminal:** in the Frontend folder, run `npm run dev`. Keep this terminal running too.
5. Open the Vite local URL, usually `http://localhost:5173/`. The page should load and request student records from the backend.

6. Enter a student name, course, and marks from 0 to 100, then click Add Student. The new record should appear in the list.

7. Try leaving a field blank or entering marks outside 0–100. The validation alert should appear and no record should be sent.

8. Open MongoDB Compass, connect to `mongodb://127.0.0.1:27017`, and check database `CollegeDB` and collection `students`.

9. If the page says it cannot load students, first check that both terminals are still running and that the backend printed the success messages.

Important limitation: this compact Master 5 code adds and lists students. For full UI-based edit/delete as in your Practical 10 handout, use the full App.jsx code from your college PDF; Master 2 already provides the matching PUT and DELETE backend routes.

MASTER 6 — React Navigation (`Frontend/src/App.jsx`)

Question covered: implement navigation between React pages.

```
// Install in Frontend first: npm install react-router-dom
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function Home() {
  return <h2>Home Page</h2>;
}

function About() {
  return <h2>About Page</h2>;
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Exact steps to perform

1. Open a terminal in the `Frontend` folder.

2. Run `npm install react-router-dom` once.

3. Open `Frontend/src/App.jsx`, replace all code with Master 6, and save.

4. Run `npm run dev` in the Frontend folder. If it is already running, leave it running.

5. Open the Vite URL in the browser. Click Home and About; the displayed page content changes without a full page reload.

All 28 Questions — Where to Look

Question	Master code / route
1. Operators and control statements in JavaScript	Master 1
2. Display a message in browser using server response	Master 2 — GET /
3. Print even numbers from 1 to 50	Master 1
4. Create different routes in Express.js	Master 2 — /home and /about
5. Add two numbers in JavaScript	Master 1
6. Send JSON response from Express	Master 2 — /json
7. Basic CSS styling to React component	Master 3
8. Create MongoDB collection and insert records	Master 2 — POST /students
9. Functional component and render in App.js	Master 3
10. Create an Express API	Master 2 — /api
11. Display current date/time dynamically in React	Master 4
12. Create GET API using Express	Master 2 — GET /students
13. Multiple React components on one page	Master 3
14. Fetch API data and display in React	Master 5
15. Pass data between React components using props	Master 3
16. Express and MongoDB API to add a user	Master 2 — adapt Student model/route to User if required
17. useState to manage/update data	Master 3
18. React form sends data to backend	Master 5
19. Form accepts input and displays it dynamically	Master 4
20. Retrieve and display MongoDB records	Master 2 + Master 5
21. Factorial of a number	Master 1
22. Insert and update records in MongoDB	Master 2 — POST and PUT
23. Arrow functions in JavaScript	Master 1
24. Store data in MongoDB	Master 2 — POST /students
25. Multiplication table	Master 1
26. React square and square root calculator	Use Master 3's useState pattern; add a number input and show n*n and Math.sqrt(n)
27. Navigation between pages in React	Master 6
28. map(), filter(), reduce()	Master 1

Troubleshooting — quick checks

‘Cannot find module’: run the install command in the correct folder (backend packages in Backend; Axios/Router in Frontend).

Port 3000 already in use: stop the other server with Ctrl+C before starting another one.

MongoDB connection failed: start the MongoDB service/server and confirm the connection string is `mongodb://127.0.0.1:27017/CollegeDB`.

React cannot fetch data: make sure the backend terminal is still running on port 3000 and the API URL matches.

Vite page not opening: run `npm run dev` from the Frontend folder and open the exact local URL printed by Vite.

After changing backend code: press Ctrl+C in the backend terminal, then run `node server.js` again. After changing React code, save the file; Vite normally refreshes automatically.